Code Implementation of Inverse Kinematics and Other Methods

Summary:

In this lesson, students write the two inverse kinematics (IK) methods from Lessons 6 and 7 as Python code, one small function at a time, and check every answer with the forward kinematics (FK) code from Lesson 5. On Day 1, they turn Lesson 6's triangles into law_of_cosines, elbow, shoulder, ik_flat and ik_3d, and handle targets that are out of reach with raise and try/except. On Day 2, they turn Lesson 7's gradient descent into error, cost, slopes and the ik_gd loop, test learning rates, and run both methods on the same targets. Then they meet the Jacobian, a table of how each joint moves the tip, see how a Jacobian method reaches the target in a few steps, and find Lesson 6's steps inside the arm's own control code. At the end, every student has a Python file that finds the joint angles for a target in two ways. Students need intermediate Python: variables, functions, lists, for loops and import, as in Lesson 5.

Objective:

By the end of this lesson, students will be able to:

- Use math.acos, math.atan2 and math.degrees to turn a number back into an angle in degrees
- Write Lesson 6's recipe as small functions (law_of_cosines, elbow, shoulder, ik_flat and ik_3d) and check each one against the answers from Lesson 6
- Check every IK answer with FK from Lesson 5
- Use raise and try/except to handle a target that is out of reach
- Write Lesson 7's gradient descent as code (error, cost, slopes and the ik_gd loop) and test different learning rates
- Run both methods on the same targets and explain why their answers can differ
- Describe what the Jacobian shows and why a Jacobian method needs fewer steps than gradient descent
- Find the steps of Lesson 6's recipe in the arm's own control code

Standards:

NGSS

- Engineering Design: HS-ETS1-2 (break a complex problem into smaller problems), HS-ETS1-3 (evaluate solutions against trade-offs), HS-ETS1-4 (use a computer model to test a solution)
- Science & Engineering Practices: Developing and Using Models; Using Mathematics and Computational Thinking; Analyzing and Interpreting Data; Engaging in Argument from Evidence
- Crosscutting Concepts: Systems and System Models; Patterns

Vocabulary:

Inverse kinematics (IK): Finding the joint angles that put the tip of the arm on a target. It is the reverse of forward kinematics (FK).

math.acos and math.atan2: Python's $\cos^{-1}$ and atan2. Like sin and cos, they work in radians.

math.degrees: Changes an angle in radians into degrees. It is the reverse of math.radians.

raise: The word that stops a function with an error message, for example when a target is out of reach.

ValueError: The kind of error Python uses for a value that does not work, such as math.acos(1.5) or a target that is too far away.

try and except: Code that tries something and, if it raises an error, runs the except part instead of stopping the program.

Default value: A value a function uses when you leave that input out. ik_gd(target) uses eta=0.1 unless you give another eta.

while loop: Code that repeats as long as a condition is true. ik_gd repeats while there are steps left and the error is not small enough.

Copy of a list: list(angles) makes a new list with the same numbers, so changing the copy does not change the original.

Unpacking: tip_3d(*angles) puts the three numbers of a list in as three inputs. angles, steps, err = ik_gd(...) takes the three things a function returns.

Test case: An input whose right answer we already know, used to check the code. The worked example (301.78, 301.78, 176.78) is our main test case.

Tolerance: How small the error must be before gradient descent stops. This lesson uses 0.5 mm, as in Lesson 7.

Jacobian: A table of how far the tip moves (in x, y and z) when each joint turns a little. A Jacobian method uses it to choose all the joint turns at once.

Resources & Materials:

- Lesson 8 slideshow, Days 1 and 2 (35 slides; the slide numbers in the Procedures are from this slideshow)
- Computers with Python 3, one per pair. IDLE, Thonny or any other editor works. No extra libraries are needed
- arm.py from Lesson 5 (each pair's own file, or the solution), in the same folder as the Lesson 8 files
- Student files: ik_starter.py (saved as ik.py), check_ik.py and jacobian_preview.py
- Solution: ik.py (the same code is on the slides, so hold the file back until students have tried it themselves). The code is in the Appendix
- Students' answers from Lessons 6 and 7, for comparing
- The assembled arm running the waypoint GUI (Code/waypoint.py), to show the arm moving to a target with its own IK (optional)
- All code uses link lengths L1 = L2 = 250 mm and the DH angles from Lessons 4 to 7: θB is the base turn, θS is measured from the floor, θE is measured from the line of the upper arm, and up

is +. If the built arm measures differently, change the two constants at the top of arm.py. The answers on the slides will change too.

Teacher note: every output on the slides and in this plan comes from running the code in the Appendix with Python 3. ik_gd uses the same settings as the computer run in Lesson 7 (start (0°, 45°, −45°), η = 0.1, tolerance 0.5 mm, at most 1000 steps), so it gives the same numbers.

Procedures:

Day 1: Lesson 6's Triangles in Python

- **Opening (4 min)** — Show the title slide and "Where Lessons 5 to 7 Left Off" (slides 1–2). In Lesson 5, `tip_3d(45, 45, -45)` gave the tip [301.78, 301.78, 176.78]. In Lesson 6, triangles turned that target back into θB = 45°, θS = 45°, θE = −45° by hand. In Lesson 7, gradient descent got within 0.5 mm, and a computer took the 84 steps. Today students write both methods in Python and check every answer with FK.
- **Setting up (3 min)** — Show "Setting Up" (slide 3). Students put arm.py from Lesson 5 in a folder, open ik_starter.py, save a copy as ik.py in the same folder and run it. If nothing prints and there is no error, it is ready. The line `from arm import L1, L2, show, flat_tip, tip_3d` brings in the link lengths, show() and the FK functions from Lesson 5.
- **From a number back to an angle (4 min)** — Show slide 4. Lesson 6 used $\cos^{-1}$ and $\tan^{-1}$. In Python they are `math.acos` and `math.atan2`, and like sin and cos they work in radians. `math.acos(0)` prints 1.5707963267948966, which is 90° in radians (π ÷ 2), and `math.degrees` changes it back: `math.degrees(math.acos(0))` prints 90.0 and `math.degrees(math.atan2(1, 1))` prints 45.0. Remind students that atan2(y, x) looks at the signs of both y and x (Lesson 6, Day 2).
- **Try It Yourself #1 (4 min)** — Students predict, then run, the three lines on slide 5. Reveal (slide 6): 60.0, 135.0 and `ValueError: math domain error`. cos 60° = 0.5. The direction (−1, 1) is up and to the left, 135°. No angle has a cosine bigger than 1, so Python stops with an error. In Lesson 6 that meant the target is out of reach.
- **Step 1: the law of cosines (5 min)** — Show slide 7. law_of_cosines(a, b, c) returns the angle opposite side c, using Lesson 6's cos C = (a² + b² − c²) ÷ (2ab). The line `cos_c = max(-1, min(1, cos_c))` keeps the value between −1 and 1, because rounding can give 1.0000001, which would crash acos. Students type it in and check `round(law_of_cosines(250, 250, 300), 2)`, which prints 73.74, Lesson 6's worked example.
- **Try It Yourself #2 (3 min)** — Students find the angle C for sides 250, 250, 250 and 250, 250, 400 (slide 8), Lesson 6's Try It Yourself #1. Reveal 60.0 and 106.26 (slide 9). In the second, the cosine is negative, so the angle is bigger than 90°.
- **Step 2: the elbow (4 min)** — Show slide 10. elbow(D) finds γ, the angle inside the triangle at the elbow, with `law_of_cosines(L1, L2, D)`. The elbow bends 180° − γ away from a straight arm, and the minus sign makes θE negative, because elbow up means the forearm bends down. Students check `elbow(461.94)`, which gives −45.0 (D = √(426.78² + 176.78²) = 461.94 for the worked example), and `elbow(353.55)`, which gives −90.0. Common mistake: returning γ = 135° as the elbow angle, as in Lesson 6.
- **Step 3: the shoulder (4 min)** — Show slide 11. shoulder(X, Y, D) adds φ, the tilt of the line D from `math.atan2(Y, X)`, and δ, the angle between D and the upper arm. δ is opposite L2, so the

call is `law_of_cosines(L1, D, L2)`. Elbow up, θS = φ + δ. Students check `shoulder(426.78, 176.78, 461.94)`, which gives 45.0 (22.5° + 22.5°).

- **Step 4: the flat arm (7 min)** — Show slides 12–13. ik_flat finds D with the Pythagorean theorem. If D is longer than both links together (500 mm), there is no triangle, so `raise ValueError("target is out of reach")` stops the code with a message. Otherwise it returns [shoulder, elbow]. Students run `show(ik_flat(426.78, 176.78))`, which prints [45.0, -45.0], then check it with FK: `show(flat_tip(45.0, -45.0))` prints [426.78, 176.78]. IK, then FK, brings us back to the target. Make this check a habit.
- **Out of reach (3 min)** — Show slide 14. `show(ik_flat(400, 400))` stops with `ValueError: target is out of reach`, because D = 565.69 mm. A robot must not move when there is no answer. To print a message and keep the program going, put the call inside `try:` and `except ValueError:`. The program then prints "out of reach".
- **Try It Yourself #3 (6 min)** — Students find the angles for (341.51, 341.51) and (250, 250), then check each with flat_tip (slide 15). These are Lesson 6's Try It Yourself #2 and #3. Reveal (slide 16): [60.0, -30.0] and [90.0, -90.0], and FK gives [341.51, 341.51] and [250.0, 250.0]. In the second, the upper arm points straight up and the forearm points straight across.
- **Step 5: the whole arm in 3D (4 min)** — Show slide 17. ik_3d is Lesson 6, Day 2: atan2 aims the base, R = $\sqrt{(x^2 + y^2)}$ is the reach across the floor, and (R, z) is a flat-arm target, so ik_flat does the rest. ik_flat already checks the reach, so ik_3d does not need its own check. Students check `show(ik_3d(301.78, 301.78, 176.78))`, which prints [45.0, 45.0, -45.0], and `show(tip_3d(45.0, 45.0, -45.0))`, which prints [301.78, 301.78, 176.78].
- **Try It Yourself #4 (6 min)** — Students run ik_3d for (295.75, 170.75, 91.51) and (−324.76, 187.50, 216.51) and check each with tip_3d (slide 18). These are Lesson 6's Try It Yourself #4 and #5. Reveal (slide 19): [30.0, 60.0, -90.0] and [150.0, 60.0, -60.0]. In the second, atan2 gives 150°; plain atan would give −30°.
- **Recap (3 min)** — Show "Day 1 Recap" (slide 20): law_of_cosines finds an angle from three sides, elbow and shoulder are Lesson 6's Steps 1 and 2, ik_flat adds a reach check, and ik_3d aims the base with atan2 and then calls ik_flat. Always check IK with FK. Tell students that tomorrow Lesson 7's gradient descent becomes code.

Day 2: Gradient Descent in Python, and the Jacobian

- **Recap (3 min)** — Show "Day 2: Where Day 1 Left Off" (slide 21). Ask: What does ik_flat(400, 400) do? (It raises a ValueError, because the target is 565.69 mm away and the arm reaches 500 mm.) Why does ik_3d use atan2 and not atan? (atan cannot tell opposite directions apart, so the base could point the wrong way.) How do we check an IK answer? (Put the angles into FK and see if the tip lands on the target.)
- **Step 6: the error and the cost (5 min)** — Show slide 22. Lesson 7's step rule uses radians, so error and cost take the angles in radians. `[math.degrees(a) for a in angles]` makes a new list with each angle in degrees, and `tip_3d(*degrees)` puts the three numbers in as three inputs. The error is the distance in 3D (the Pythagorean theorem), and the cost is Lesson 7's C = (error ÷ 250)².
- **Test the error (3 min)** — Show slide 23. With the worked example target and the start (0°, 45°, −45°) changed to radians, `error` prints 326.64 and `cost` prints 1.71. The start pose puts the tip

at (426.78, 0, 176.78), 326.64 mm from the target, which is the first row of Lesson 7's "Watching the Error Fall", and (326.64 ÷ 250)² = 1.71.

- **Step 7: the slopes (6 min)** — Show slide 24. slopes nudges one joint at a time, as in Lesson 7. `list(angles)` makes a copy, so the other joints stay where they are. The code nudges up and down by h and divides by 2h, which is fairer than nudging one side only (Lesson 7 nudged one side by hand). h = 0.0001 rad is a tiny turn. Students check `show(slopes(start, target))`, which prints [-4.12, -0.71, 0.0]. The negative slopes mean θB and θS must get bigger.
- **Step 8: the loop (7 min)** — Show slide 25 and walk through ik_gd. The inputs start, eta, tol and max_steps have default values, which are Lesson 7's settings. The angles are changed to radians, and `while` repeats as long as there are steps left and the error is not small enough yet. Inside, each joint takes Lesson 7's step θ_new = θ − η × slope. At the end the angles go back to degrees, and `return` sends back three things: the angles, the steps used and the error that is left.
- **Run the worked example (4 min)** — Show slide 26. `angles, steps, err = ik_gd((301.78, 301.78, 176.78))` takes the three things ik_gd returns. It prints [45.0, 45.15, -45.29], then 84 0.49, and FK puts the tip at [301.45, 301.45, 176.63]. These are the numbers the computer gave in Lesson 7: 84 steps, 0.49 mm from the target. Lesson 6's exact answer is (45°, 45°, −45°).
- **Try It Yourself #5 (5 min)** — Students run the worked example with eta=0.02, eta=0.2 and eta=0.3 and guess the steps first (slide 27). Reveal (slide 28): 364 steps for 0.02, 84 for 0.1 and 52 for 0.2, all ending under 0.5 mm. With 0.3 the loop runs all 1000 steps and ends 288.01 mm away: the step is too big, so the error bounces between 234.5 and 288.0 mm and never settles, as in Lesson 7. ik_gd stops at the step limit, so always look at the error.
- **Test both methods (6 min)** — Show slides 29–30. check_ik.py runs both methods on four targets from Lesson 6. `try` and `except` catch the out-of-reach error so the loop keeps going, and `end=" "` keeps show() on the same line. Students run it and fill in the table. Rows 1 and 2 give the same pose (gradient descent is close, not exact). In row 3, (−324.76, 187.50, 216.51), trigonometry gives [150.0, 60.0, -60.0] and gradient descent gives [-30.0, 179.88, -59.78] in 179 steps: both are right, because gradient descent found the other valley, with the base pointing away and the arm reaching back over the top (Lesson 7). In row 4, (400, 300, 200), trigonometry stops with an error, while gradient descent ends 38.52 mm short after 1000 steps with no error. Always check the error.
- **The Jacobian (5 min)** — Show slide 31. Ask: how far does the tip move when one joint turns by 1°? Nudge that joint and run FK, as in Lesson 7. At the pose (45°, 45°, −45°), turning θB by 1° moves the tip (−5.27, 5.27, 0.00) mm, turning θS moves it (−2.18, −2.18, 7.45) mm and turning θE moves it (0.00, 0.00, 4.36) mm. This table is called the Jacobian. θB swings the tip around the base. θE lifts it straight up, because the forearm points straight across in this pose (250 mm × 1° in radians = 4.36 mm). Teacher note: in college courses calculus gives this table exactly; here it comes from nudging.
- **A Jacobian method (5 min)** — Show slide 32 and run jacobian_preview.py. Gradient descent only asks which way is downhill and takes a small step. A Jacobian method uses the table to work out the turns of all three joints that move the tip straight to the target. It prints the table, then [45.0, 45.0, -45.0] in 5 steps (0.01 mm), against 84 steps for gradient descent. For (295.75, 170.75, 91.51) it takes 4 steps against 79, and for (−125, 216.51, 250) it takes 11 against 73. Students do not need to write this code. It limits each turn to 20° so it never jumps too far. It has limits too:

near a straight arm the table cannot push the tip farther out, and out of reach it also ends short, so real robots add safety steps.

- **IK inside the arm's own code (5 min)** — Show slide 33: the inverse_kinematics function from Code/waypoint.py, the arm's control program. Pairs find the base angle (`atan2(y, x)`), the reach (`hypot`), the reach check (`return []` when the target is too far), the elbow (the law of cosines) and the shoulder (`alpha` plus another law of cosines). In this file L2 and L3 are the two links, 0.22 m each, and L1 is the height of the shoulder. It works in metres and radians and measures the shoulder from straight up, but the steps are Lesson 6's. If the arm is set up, show the waypoint GUI moving the arm to a target (optional).
- **Three ways to do IK (4 min)** — Show slide 34. Trigonometry is exact in one pass but needs triangles for this arm and stops with an error when the target is out of reach. Gradient descent needs only FK and the same loop works for any arm, but it is close, not exact, and can take 50 to 1000 steps. A Jacobian method also works for any arm and takes about 4 to 20 steps. Ask: if the arm had six joints, which method would you use, and why? Whichever method is chosen, check the answer with FK and look at the error.
- **Wrap-up (2 min)** — Show "What We Learned" (slide 35): each step of Lesson 6's recipe became a small function, Lesson 7's loop became error, cost, slopes and ik_gd, check IK with FK every time, raise and try/except handle targets out of reach, and the Jacobian makes IK faster. Preview Lesson 9: AI object detection, so the arm can find its own targets.

Assessment:

Assess with working code. On Day 1, each pair shows `ik_flat(426.78, 176.78)` printing [45.0, -45.0] with the FK check, and their answers to Try It Yourself #2 to #4. On Day 2, pairs show `ik_gd` printing 84 steps and 0.49 mm for the worked example, their table for Try It Yourself #5 and the output of check_ik.py. Students check each answer against the solution on the slide, and can check #2 to #5 by running their own code.

Use these questions as an exit ticket or short quiz. The answers are in red.

- Why does law_of_cosines use math.degrees after math.acos?
  Answer: math.acos gives the angle in radians. Without math.degrees, law_of_cosines(250, 250, 300) would return 1.29 instead of 73.74.

- What does law_of_cosines(250, 250, 353.55) return? What does elbow(353.55) return?
  Answer: 90.0, so the elbow bends 180° − 90° = 90° and elbow(353.55) returns −90.0.

- A student's elbow returns 180 - gamma. What goes wrong, and how does FK show it?
  Answer: The minus sign is missing, so the elbow bends up. ik_flat(426.78, 176.78) gives [45.0, 45.0], and flat_tip(45, 45) gives [176.78, 426.78] instead of the target.

- Why does ik_3d use math.atan2(y, x) instead of math.atan(y / x)?
  Answer: atan cannot tell opposite directions apart (and fails when x = 0). For (−324.76, 187.50, 216.51) atan gives −30°, so the base points the opposite way; atan2 gives 150°.

- What happens when you run ik_flat(400, 400)? How can a program keep going?

Answer: D = 565.69 mm is more than 500 mm, so ik_flat raises ValueError: target is out of reach. Put the call inside try and except ValueError and print a message instead.

- In slopes, why do we copy the list with list(angles) before nudging a joint?
  Answer: So that nudging one joint does not change the original angles. The other joints stay where they are, and every nudge starts from the same pose.

- ik_gd((301.78, 301.78, 176.78), eta=0.3) stops after 1000 steps with an error of 288.01 mm. What happened, and what should you do?
  Answer: The learning rate is too big, so the steps overshoot and the error bounces without settling. Use a smaller η, such as 0.1 (84 steps), and always check the error.

- For (−324.76, 187.50, 216.51), trigonometry gives [150.0, 60.0, -60.0] and gradient descent gives [-30.0, 179.88, -59.78]. Is one of them wrong?
  Answer: No. Both put the tip on the target. Gradient descent found the other valley, with the base pointing away and the arm reaching back over the top. Choose the answer that fits the arm's joint limits.

- What does the Jacobian tell you? Use the θE row at the pose (45°, 45°, −45°).
  Answer: How far the tip moves in x, y and z when each joint turns by 1°. The θE row (0.00, 0.00, 4.36) means turning the elbow up by 1° lifts the tip 4.36 mm straight up.

- Why does a Jacobian method need fewer steps than gradient descent?
  Answer: It uses the table to choose the turns of all three joints that move the tip straight toward the target, instead of small downhill steps. For the worked example it takes 5 steps instead of 84.

Appendix: ik.py (solution)

Save this code as ik.py in the same folder as arm.py from Lesson 5. Students start from ik_starter.py, which has only the first lines and the list of steps. No extra libraries are needed.

```
"""Lesson 8: inverse kinematics in Python.

The two methods from Lessons 6 and 7, written as code:
    Day 1  trigonometry: the law of cosines (Lesson 6)
    Day 2  gradient descent: guess, measure, improve (Lesson 7)

Angles are in degrees, measured the DH way, as in arm.py from Lesson 5:
    theta_b  the base turn, measured from the x axis
    theta_s  the upper arm, measured from the floor
    theta_e  the forearm, measured from the line of the upper arm
Up is +, so elbow up means theta_e is negative. Lengths are in mm.
Save this file in the same folder as arm.py.
"""
import math
from arm import L1, L2, show, flat_tip, tip_3d


# ---------- Day 1: trigonometry (Lesson 6) ----------

# Step 1
def law_of_cosines(a, b, c):
    """The angle (degrees) opposite side c in a triangle with sides a, b, c."""
```

```
    cos_c = (a ** 2 + b ** 2 - c ** 2) / (2 * a * b)
    cos_c = max(-1, min(1, cos_c))      # rounding can give 1.0000001
    return math.degrees(math.acos(cos_c))


# Step 2
def elbow(D):
    """The elbow angle theta_e for a target D mm away (elbow up)."""
    gamma = law_of_cosines(L1, L2, D)   # the angle inside the triangle
    return -(180 - gamma)               # the bend, down is negative


# Step 3
def shoulder(X, Y, D):
    """The shoulder angle theta_s for the target (X, Y) (elbow up)."""
    phi = math.degrees(math.atan2(Y, X))    # the tilt of the line D
    delta = law_of_cosines(L1, D, L2)       # between D and the upper arm
    return phi + delta


# Step 4
def ik_flat(X, Y):
    """The flat arm: target (X, Y) -> [theta_s, theta_e]."""
    D = math.sqrt(X ** 2 + Y ** 2)
    if D > L1 + L2:
        raise ValueError("target is out of reach")
    return [shoulder(X, Y, D), elbow(D)]


# Step 5
def ik_3d(x, y, z):
    """The 3D arm: target (x, y, z) -> [theta_b, theta_s, theta_e]."""
    theta_b = math.degrees(math.atan2(y, x))   # aim the base
    R = math.sqrt(x ** 2 + y ** 2)             # flatten: reach across the floor
    theta_s, theta_e = ik_flat(R, z)           # the flat arm does the rest
    return [theta_b, theta_s, theta_e]


# ---------- Day 2: gradient descent (Lesson 7) ----------
# The step rule uses radians, so inside the loop the angles are in radians.

# Step 6
def error(angles, target):
    """How far (mm) the tip is from the target. angles in radians."""
    degrees = [math.degrees(a) for a in angles]
    tip = tip_3d(*degrees)
    total = 0
    for i in range(3):
        total += (tip[i] - target[i]) ** 2
    return math.sqrt(total)


def cost(angles, target):
    """The cost from Lesson 7: C = (error / 250) ** 2."""
    return (error(angles, target) / L1) ** 2


# Step 7
```

```
def slopes(angles, target, h=0.0001):
    """One slope per joint: nudge that joint up and down by h radians."""
    result = []
    for i in range(3):
        up = list(angles)
        up[i] += h
        down = list(angles)
        down[i] -= h
        result.append((cost(up, target) - cost(down, target)) / (2 * h))
    return result


# Step 8
def ik_gd(target, start=(0, 45, -45), eta=0.1, tol=0.5, max_steps=1000):
    """Gradient descent IK. Returns the angles (degrees), the steps used
    and the error that is left (mm)."""
    angles = [math.radians(a) for a in start]
    steps = 0
    while steps < max_steps and error(angles, target) >= tol:
        g = slopes(angles, target)
        for i in range(3):
            angles[i] = angles[i] - eta * g[i]    # one step downhill
        steps += 1
    degrees = [math.degrees(a) for a in angles]
    return degrees, steps, error(angles, target)


if __name__ == "__main__":
    show(ik_flat(426.78, 176.78))                 # [45.0, -45.0]
    show(ik_3d(301.78, 301.78, 176.78))           # [45.0, 45.0, -45.0]
    angles, steps, err = ik_gd((301.78, 301.78, 176.78))
    show(angles)                                  # [45.0, 45.15, -45.29]
    print(steps, round(err, 2))                   # 84 0.49
```

Day 2 “Test Both Methods” script, check_ik.py. It prints the table on the “What the Tests Show” slide.

```
"""Lesson 8, Day 2: test both IK methods on the same targets.

Put this file next to arm.py and ik.py and run:  python check_ik.py
"""
from arm import show, tip_3d
from ik import ik_3d, ik_gd

targets = [(301.78, 301.78, 176.78),     # the worked example
           (295.75, 170.75, 91.51),      # Lesson 6, Try It Yourself #4
           (-324.76, 187.50, 216.51),    # Lesson 6, Try It Yourself #5
           (400, 300, 200)]              # out of reach

for target in targets:
    print("target:", target)
    try:
        angles = ik_3d(*target)
        print("  trig:", end=" ")
        show(angles)
    except ValueError:
        print("  trig: out of reach")
    angles, steps, err = ik_gd(target)
    print("  gd:  ", end=" ")
    show(angles)
    print("        ", steps, "steps, error", round(err, 2), "mm")
```

Day 2 Jacobian preview, jacobian_preview.py. Students run it but do not need to write it. It prints the table on “The Jacobian” slide, then 5 steps (0.01 mm) for the Jacobian method and 84 steps (0.49 mm) for gradient descent.

```
"""Lesson 8 preview: the Jacobian.

The Jacobian is a table that says how far the tip moves when each joint
turns by 1 degree. A Jacobian method uses that table to choose all three
joint changes at once, so it needs far fewer steps than gradient descent.
You do not need to write this code. Run it and compare the step counts.
Save it in the same folder as arm.py and ik.py.
"""
import math
from arm import show, tip_3d
from ik import ik_gd


def jacobian(angles, h=0.001):
    """One column per joint: how the tip (x, y, z) moves, in mm per degree."""
    tip = tip_3d(*angles)
    columns = []
    for i in range(3):
        nudged = list(angles)
        nudged[i] += h
        moved = tip_3d(*nudged)
        columns.append([(moved[k] - tip[k]) / h for k in range(3)])
    return columns


def det3(a, b, c):
    """A number made from three columns (the determinant). 0 means stuck."""
    return (a[0] * (b[1] * c[2] - b[2] * c[1])
            - b[0] * (a[1] * c[2] - a[2] * c[1])
            + c[0] * (a[1] * b[2] - a[2] * b[1]))


def solve(columns, move):
    """The joint changes (degrees) that move the tip by move."""
    a, b, c = columns
    d = det3(a, b, c)
    return [det3(move, b, c) / d, det3(a, move, c) / d, det3(a, b, move) / d]


def ik_jacobian(target, start=(0, 45, -45), tol=0.5, max_steps=100,
                max_turn=20):
    """Jacobian IK: each step aims the tip straight at the target."""
    angles = list(start)
    steps = 0
    while steps < max_steps:
        tip = tip_3d(*angles)
        move = [target[k] - tip[k] for k in range(3)]
        error = math.sqrt(move[0] ** 2 + move[1] ** 2 + move[2] ** 2)
        if error < tol:
            break
        change = solve(jacobian(angles), move)
        biggest = max(abs(c) for c in change)
        if biggest > max_turn:              # never turn a joint too far at once
            change = [c * max_turn / biggest for c in change]
        for i in range(3):
```

```
            angles[i] += change[i]
        steps += 1
    return angles, steps, error


if __name__ == "__main__":
    for column in jacobian([45, 45, -45]):
        show(column)            # how the tip moves per degree of each joint

    target = (301.78, 301.78, 176.78)
    angles, steps, err = ik_jacobian(target)
    show(angles)
    print("Jacobian:", steps, "steps,", round(err, 2), "mm")
    angles, steps, err = ik_gd(target)
    print("gradient descent:", steps, "steps,", round(err, 2), "mm")
```